



CSE 585: Post-Training Lecture Series

Sep. 30, 2025

Esha Munagala, Alvaro Vega Hidalgo, Chimaobi Okite

Post-Training

```
graph TD; A[Post-Training] --> B[Supervised Fine-tuning]; A --> C[RL]; C --> D["AI-Alignment<br/>Reinforcement learning with<br/>Human Feedback (RLHF)"]; C --> E["Reasoning<br/>RL for reasoning eg<br/>PRM + RL"]; D --> F["HybridFlow: A Flexible and Efficient<br/>RLHF Framework. Guangming Sheng, Chi<br/>Zhang, Zilingfeng Ye, Xibin Wu, Wang<br/>Zhang, Ru Zhang, Yanghua Peng, Haibin<br/>Lin, Chuan Wu (EuroSys '25)"]; E --> G["AREAL: A Large-Scale Asynchronous<br/>Reinforcement Learning System for<br/>Language Reasoning. Wei Fu, Jiaxuan Gao,<br/>Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He,<br/>Shusheng Xu, Guo Wei, Jun Mei, Jiashu<br/>Wang, Tongkai Yang, Binhang Yuan, Yi Wu"];
```

Supervised Fine-tuning

RL

AI-Alignment

Reinforcement learning with
Human Feedback (RLHF)

Reasoning

RL for reasoning eg
PRM + RL

**HybridFlow: A Flexible and Efficient
RLHF Framework.** Guangming Sheng, Chi
Zhang, Zilingfeng Ye, Xibin Wu, Wang
Zhang, Ru Zhang, Yanghua Peng, Haibin
Lin, Chuan Wu (EuroSys '25)

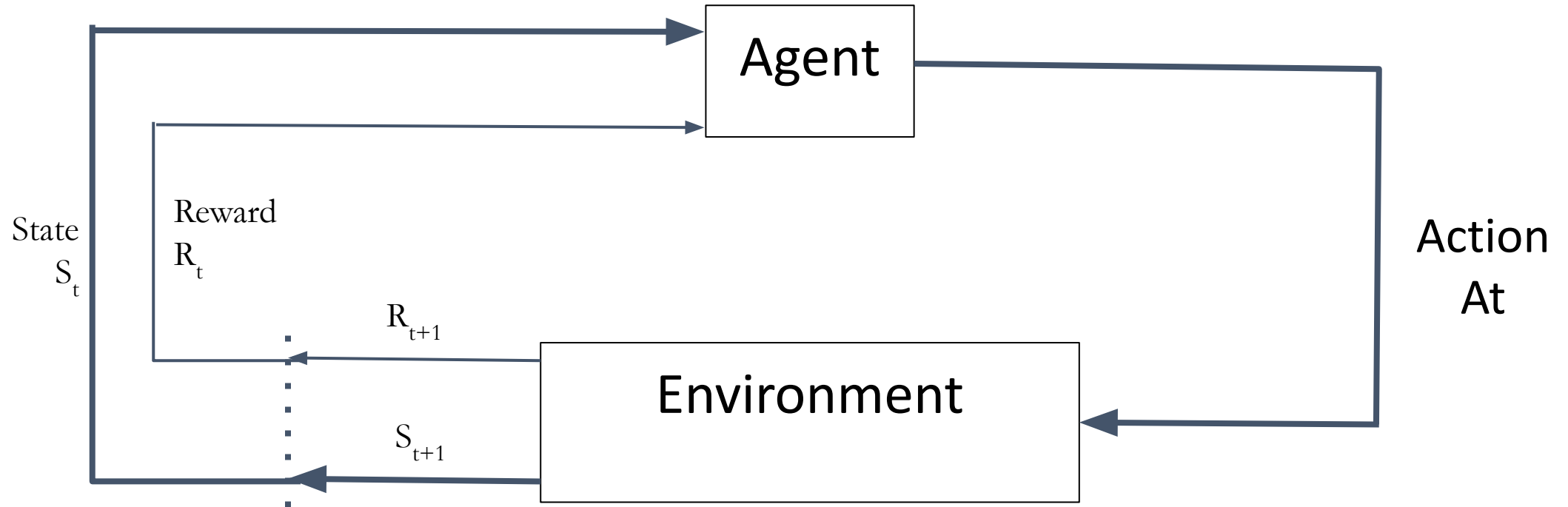
**AREAL: A Large-Scale Asynchronous
Reinforcement Learning System for
Language Reasoning.** Wei Fu, Jiaxuan Gao,
Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He,
Shusheng Xu, Guo Wei, Jun Mei, Jiashu
Wang, Tongkai Yang, Binhang Yuan, Yi Wu

Background

- Reinforcement Learning (RL)
- RL in the era of LLMs

What is RL?

Reinforcement learning is the process of learning from interaction



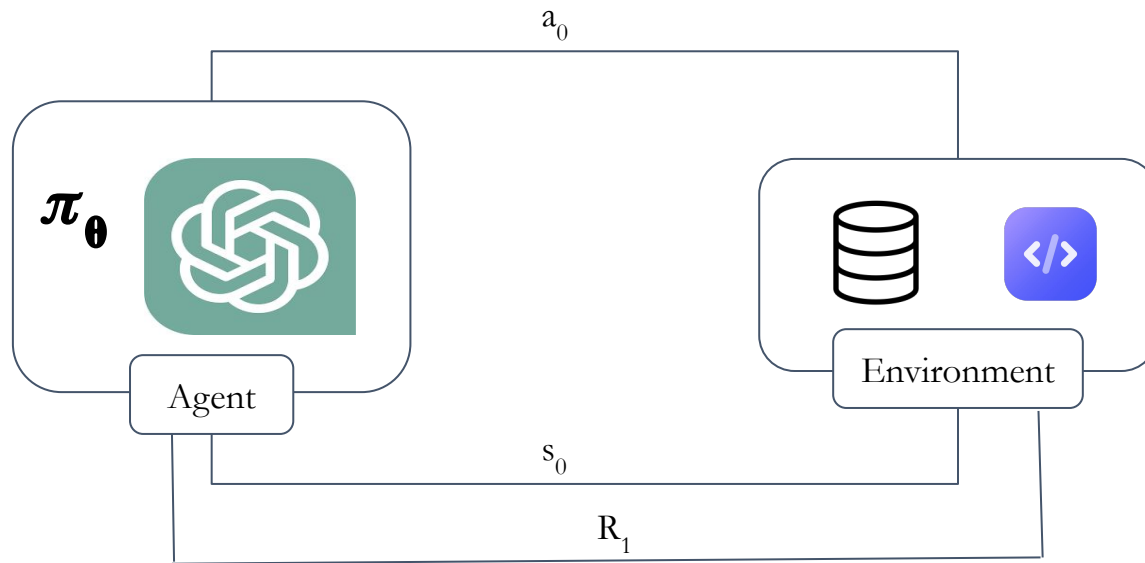
Agent's Goal?

To learn a policy π , that maximizes G_t .

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

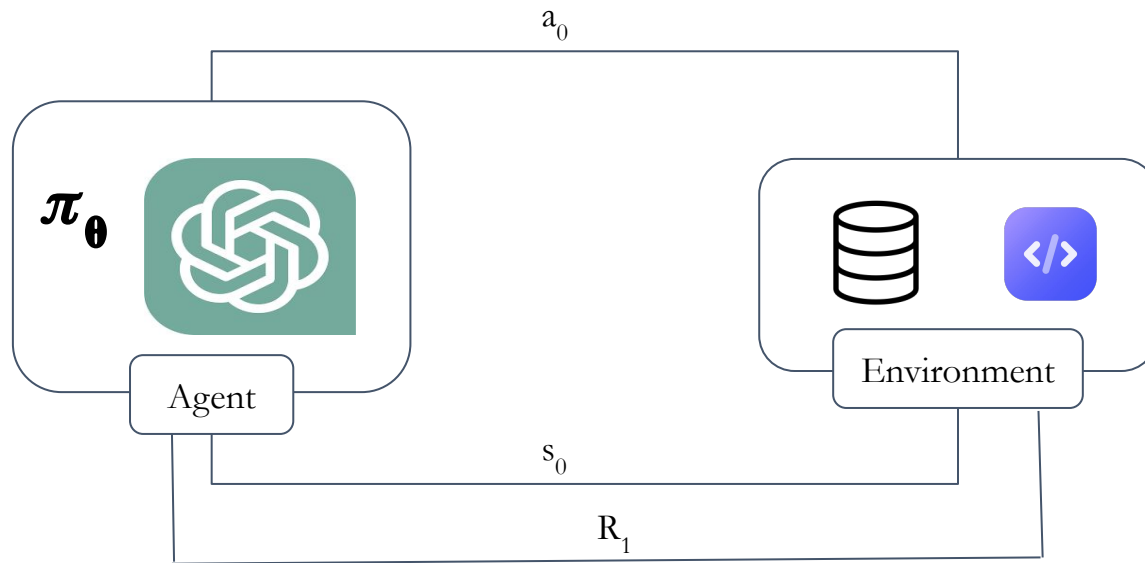


Preliminaries about RL Training



s_0 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages?

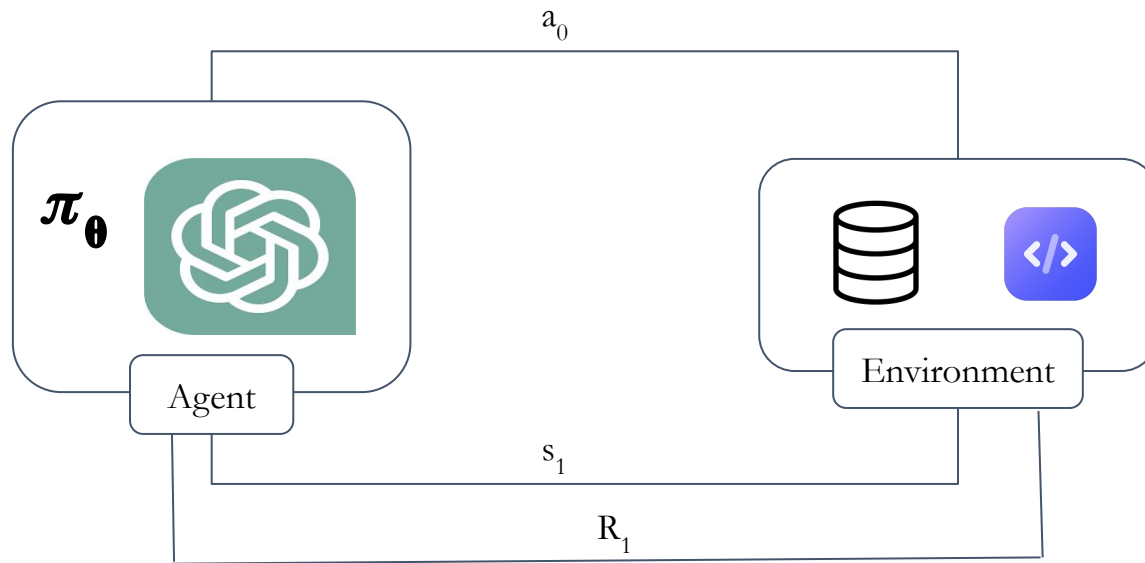
Preliminaries about RL Training



s_0 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages?

a_0 : In

Preliminaries about RL Training

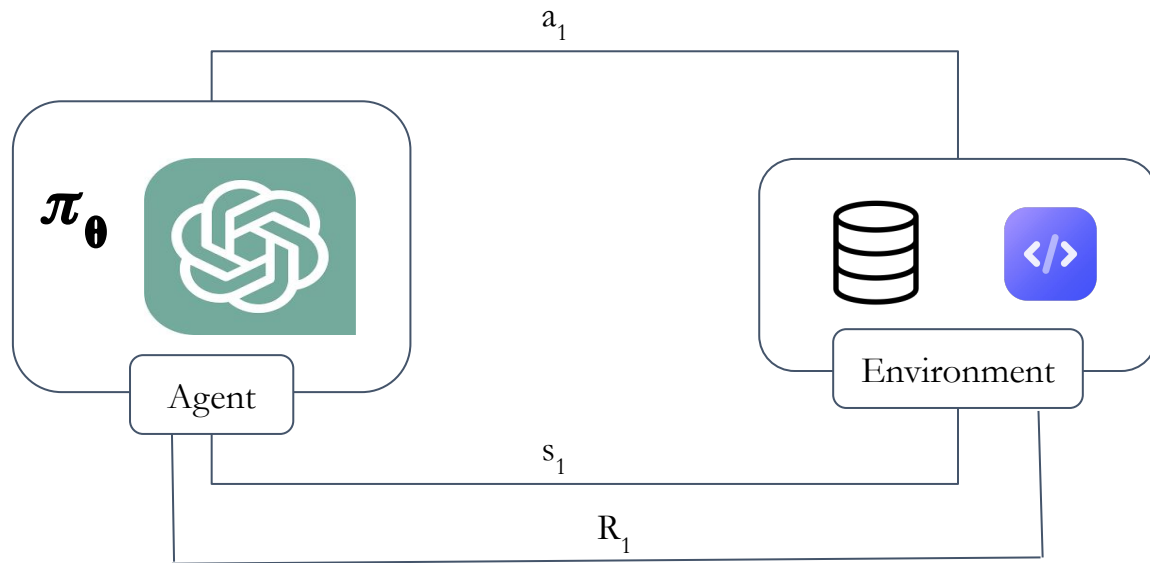


s_0 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages?

a_0 : In

s_1 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In

Preliminaries about RL Training



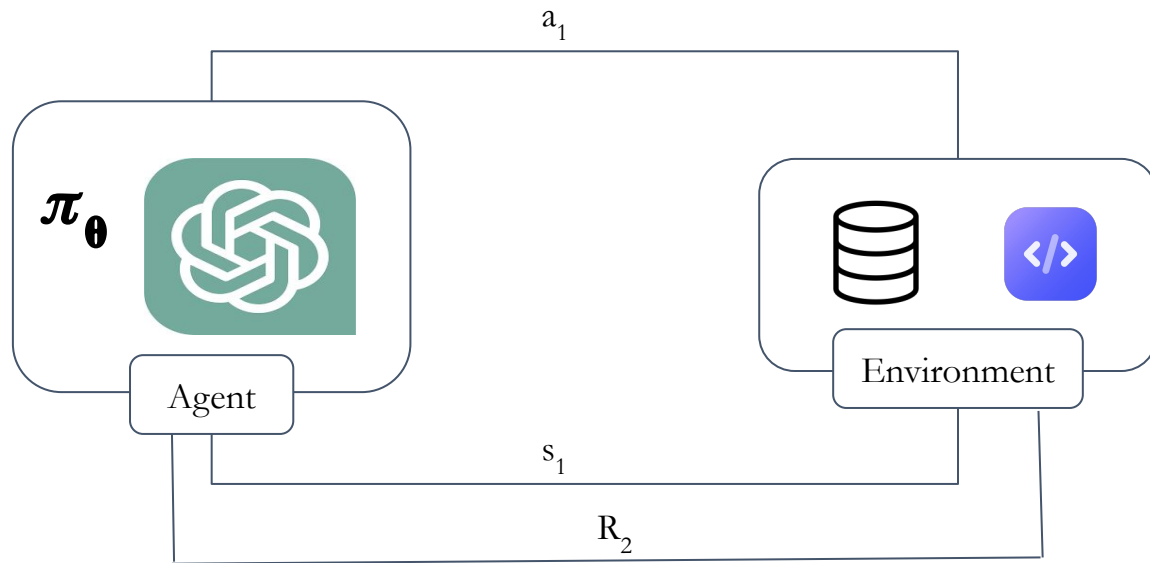
s_0 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages?

a_0 : In

s_1 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In

a_1 : one

Preliminaries about RL Training



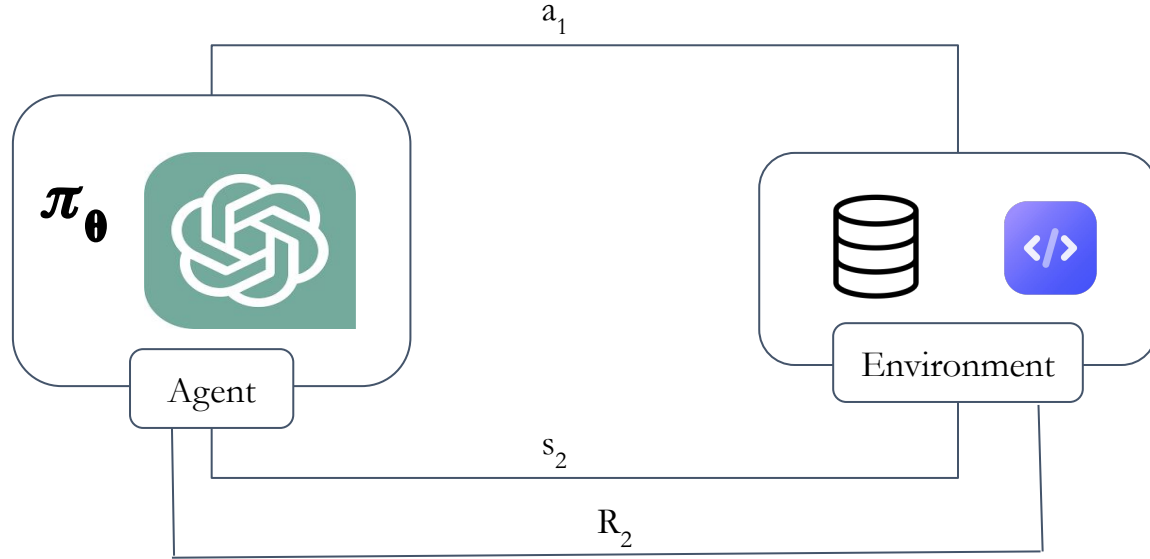
s_0 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages?

a_0 : In

s_1 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In

a_1 : one

Preliminaries about RL Training



s_0 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages?

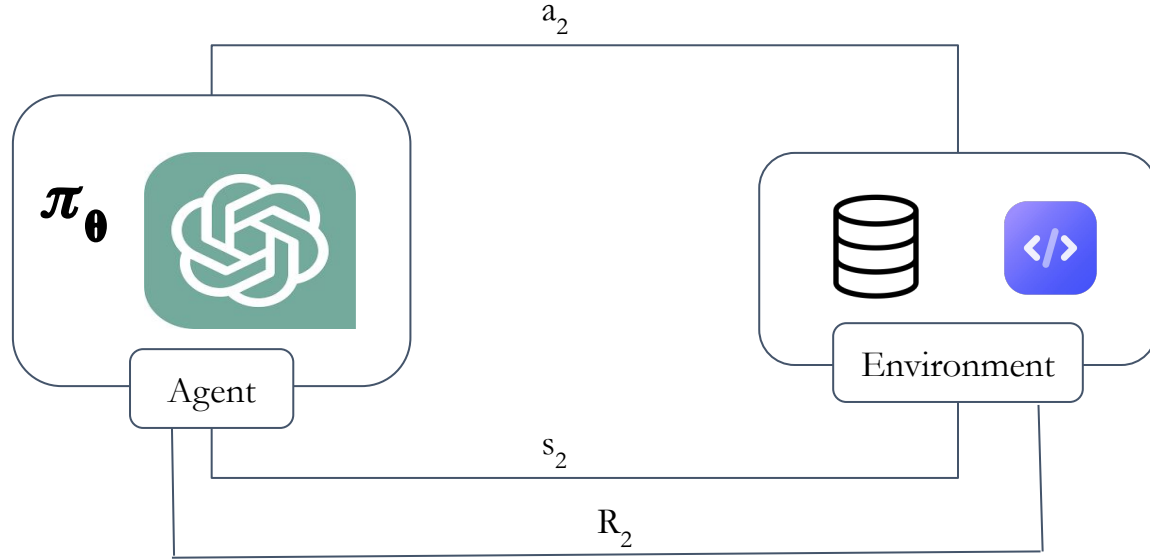
a_0 : In

s_1 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In

a_1 : one

s_2 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In one

Preliminaries about RL Training



s_0 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages?

a_0 : In

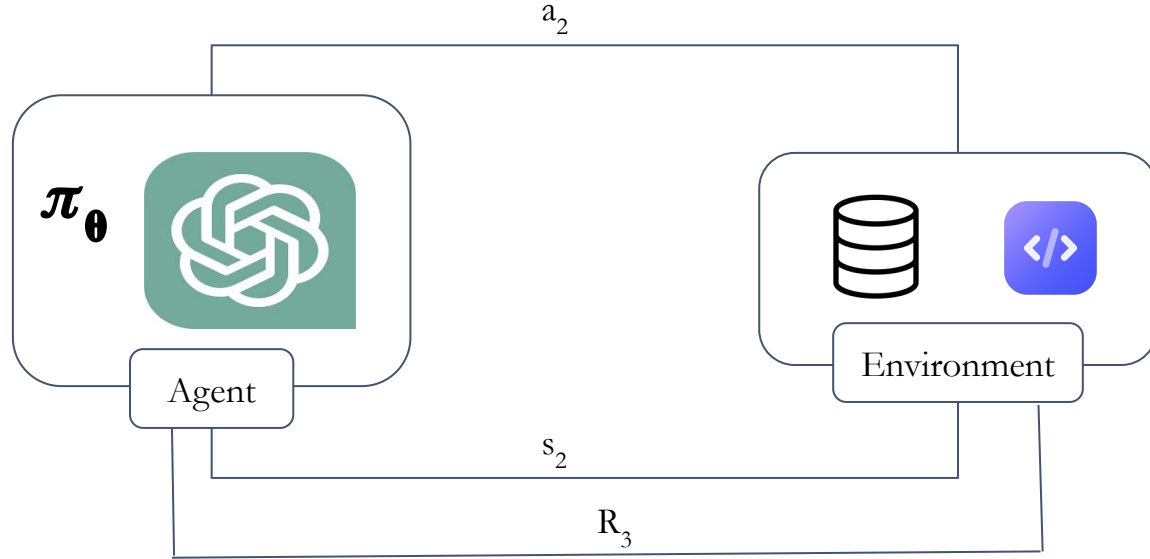
s_1 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In

a_1 : one

s_2 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In one

a_2 : one hour

Preliminaries about RL Training



s_0 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages?

a_0 : In

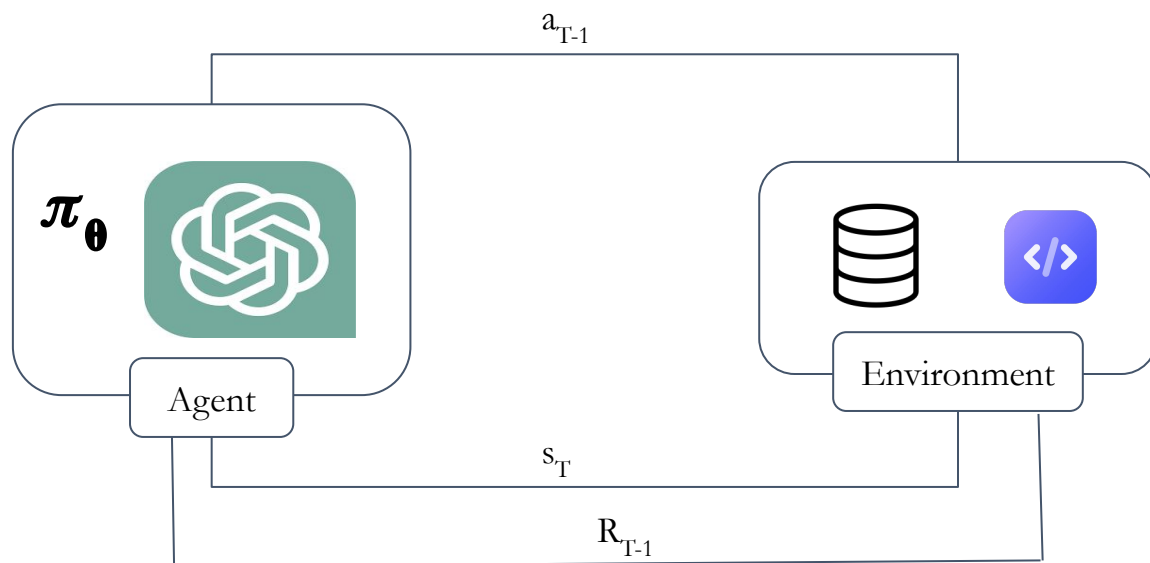
s_1 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In

a_1 : one

s_2 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In one

a_2 : one hour

Preliminaries about RL Training



s_0 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages?

a_0 : In

s_1 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In

a_1 : one

.

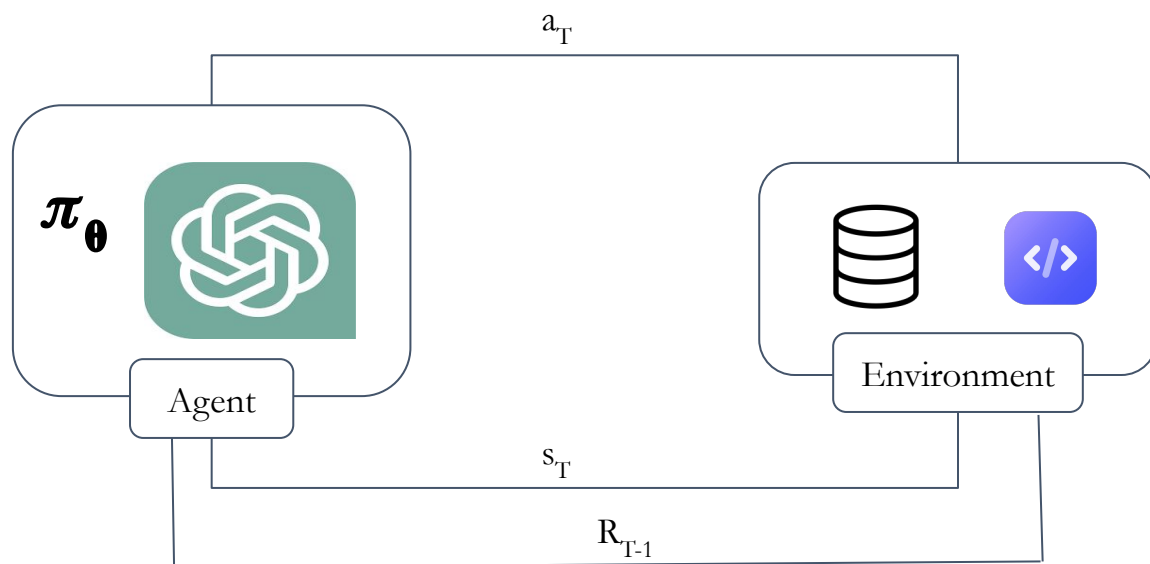
s_T : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In one hour, there are 3 sets of 20 minutes.

...

It will take her $120/24 = \ll 120/24=5 \gg 5$ hours to read 120 pages.

####

Preliminaries about RL Training



s_0 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages?

a_0 : In

s_1 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In

a_1 : one

.

s_T : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In one hour, there are 3 sets of 20 minutes.

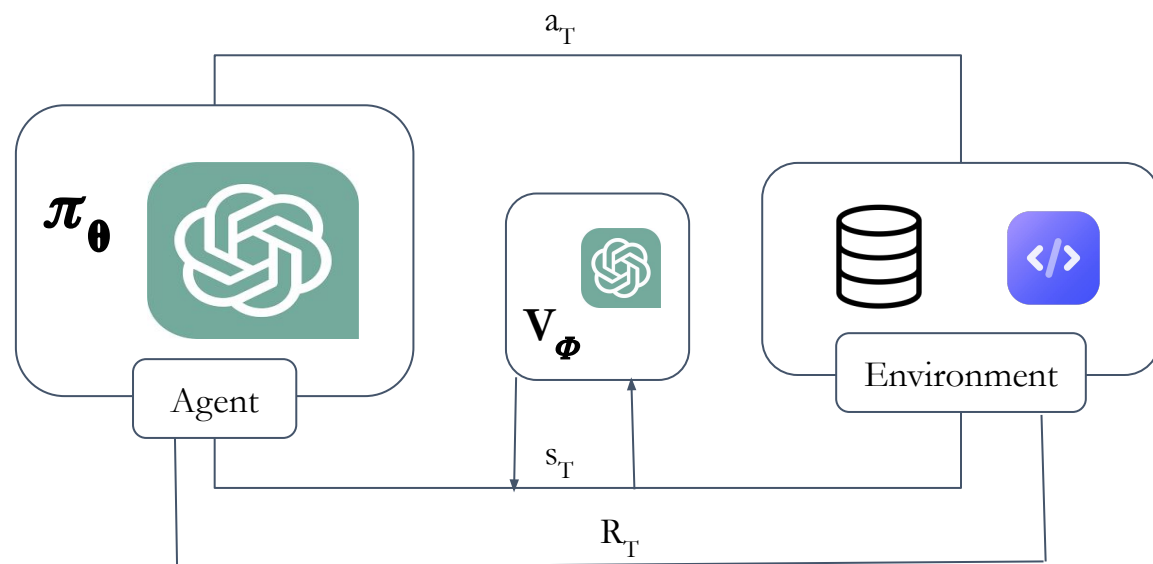
...

It will take her $120/24 = \langle\langle 120/24=5 \rangle\rangle 5$ hours to read 120 pages.

####

a_T : 5

Preliminaries about RL Training



$$J(\theta) = \mathbb{E}_{q \sim \mathcal{D}, a_t \sim \pi_\theta(\cdot | q, a_{<t})} \left[\sum_{t=1}^H \gamma^{t-1} r(s_t, a_t) \right]$$

$$\hat{A}(s_t, a_t) = r(s_t, a_t) + V(s_{t+1}) - V(s_t)$$

$$J(\theta) \approx \mathbb{E}_{q \sim \mathcal{D}, a_t \sim \pi_\theta(\cdot | q, a_{<t})} \left[\sum_{t=1}^H \log \pi_\theta(a_t | s_t) \hat{A}(s_t, a_t) \right]$$

s_0 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages?

a_0 : In

s_1 : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In

a_1 : one

...

s_T : Joy can read 8 pages of a book in 20 minutes. How many hours will it take her to read 120 pages? In one hour, there are 3 sets of 20 minutes.

...

It will take her $120/24 = 5$ hours to read 120 pages.

####

a_T : 5

Preliminaries about RL Training

Algorithm 1 Minimal RL Training Loop

- 1: Initialize policy function θ_0 and value function ϕ_0
 - 2: **for** $i = 0$ to n **do**
 - 3: Generate an episode $(S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T)$ following $\pi_i(\cdot|\cdot, \theta)$
 - 4: **for** $epoch = 0$ to n **do**
 - 5: Compute advantages A_t
 - 6: Update policy parameters θ
 - 7: Update value function parameters ϕ
 - 8: **end for**
 - 9: **end for**
-

$$J_{\text{PPO}}(\theta) = \mathbb{E}_{q \sim \mathcal{D}, a_t \sim \pi_{\text{old}}(\cdot|q, a_{<t})} \left[\sum_{t=1}^H \min \left(u_t(\theta) \hat{A}(s_t, a_t), \text{clip}(u_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}(s_t, a_t) \right) \right],$$

Here $U_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{\text{old}}(a_t | s_t)$



**AREAL: A Large-Scale Asynchronous
Reinforcement Learning System for
Language Reasoning**

Why AReal? Related Works

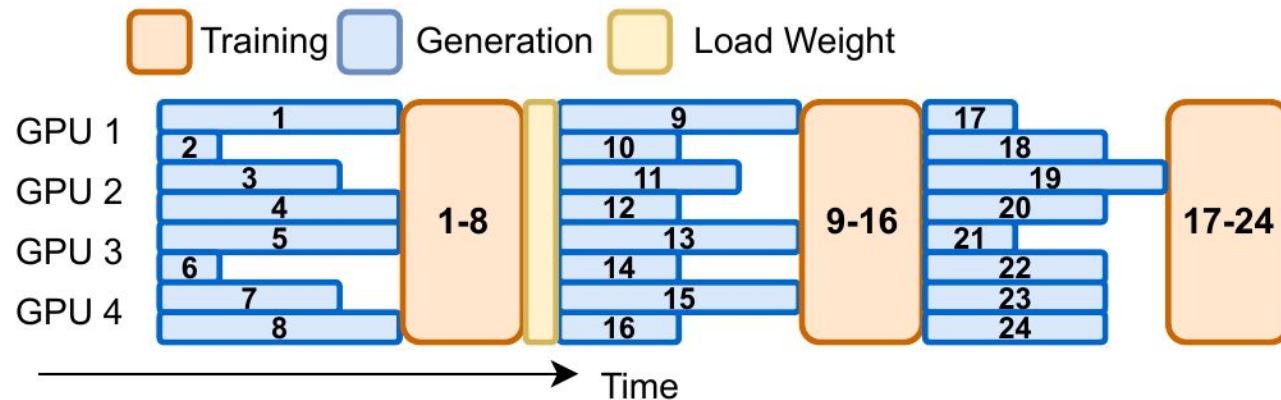


Fig: Execution Timeline of a Synchronous RL System

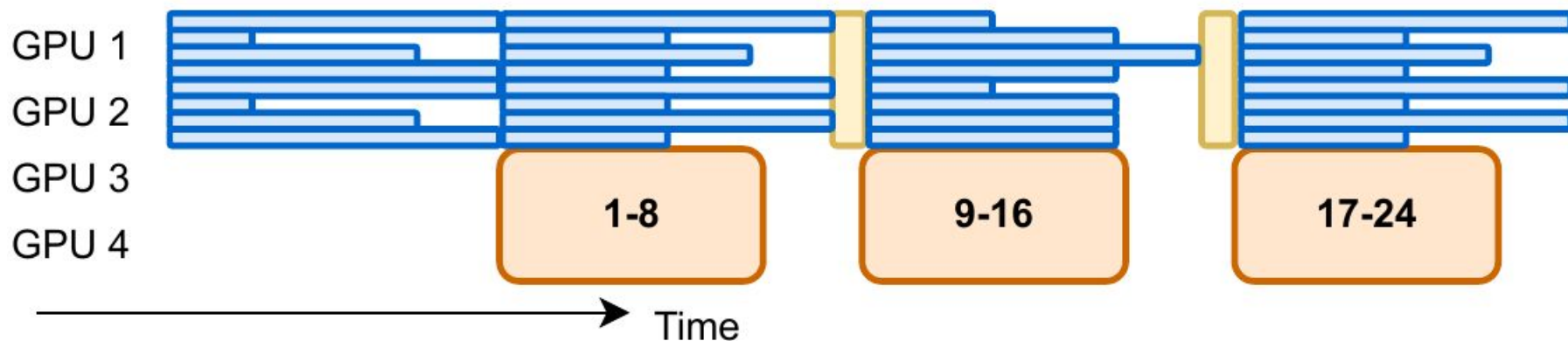


Fig: Execution Timeline of a Synchronous RL System with one time step overlap

Issues?

- Inference devices are underutilized
- Synchronous RL systems exhibit poor scalability across compute

Solution?

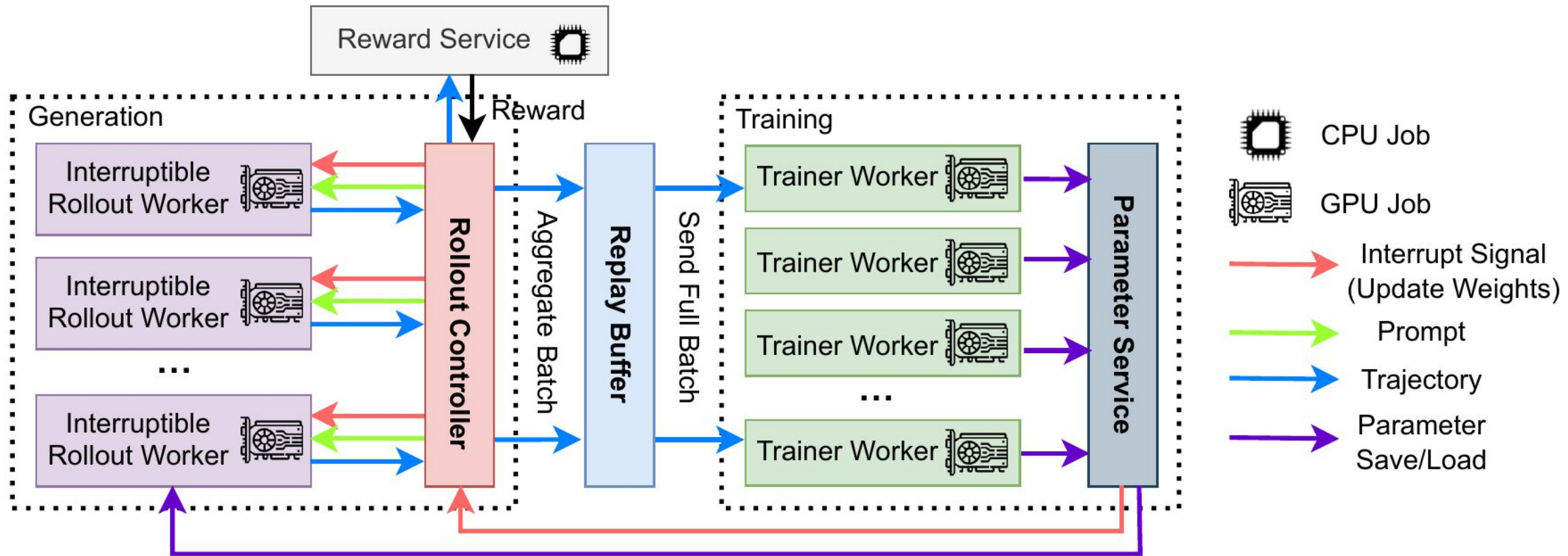
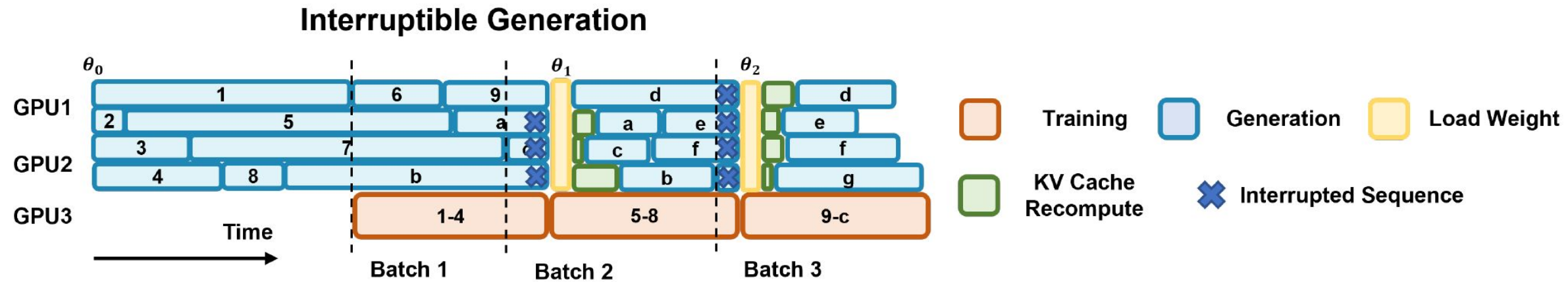


Fig: The AREAL architecture featuring asynchronous generation and training components.

Solution?



This isn't the perfect straightford
solution we hoped for 🥵
Can you identify the issue?

Algorithm Challenges

Data Staleness:

- Batch can have data from multiple prior policy

Solution:

Staleness-Aware Training

$$\lfloor (N_r - 1)/B \rfloor \leq i + \eta$$

N_r = Total no. of generated trajectories

i = current policy

B = Batch size and η = Hyperparameter



Algorithm Challenges

Data Staleness:

- Batch can have data from multiple prior policy

Solution:

Staleness-Aware Training

- introduce a Hyperparameter (n) that control how far apart the data generation model should be from our current policy



Algorithm Challenges

Inconsistent Policy Versions:

- Normal PPO $\rightarrow$ all trajectories are generated from a single policy π_{old}
- Here? We have actions within a Horizon generated from multiple policies

Solution:

Decoupled PPO Objective

$$J(\theta) = \mathbb{E}_{q \sim \mathcal{D}, a_t \sim \pi_{\text{behav}}} \left[\sum_{t=1}^H \min \left(\underbrace{\frac{\pi_{\theta}}{\pi_{\text{behav}}}}_{\text{Importance Ratio}} \hat{A}_t, \overbrace{\frac{\pi_{\text{prox}}}{\pi_{\text{behav}}} \text{clip} \left(\underbrace{\frac{\pi_{\theta}}{\pi_{\text{prox}}}}_{\text{Trust Region Center}}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t}_{\text{Importance Ratio}} \right) \right] \quad (4)$$

$$= \mathbb{E}_{q \sim \mathcal{D}, a_t \sim \pi_{\text{behav}}} \left[\sum_{t=1}^H \frac{\pi_{\text{prox}}}{\pi_{\text{behav}}} \min \left(u_t^{\text{prox}}(\theta) \hat{A}_t, \text{clip} \left(u_t^{\text{prox}}(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right], \quad (5)$$

$$u_t^{\text{prox}}(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\text{prox}}(a_t | s_t)}$$

Algorithm Challenges

Inconsistent Policy Versions:

- Normal PPO $\rightarrow$ all trajectories are generated from a single policy π_{old}
- Here? We have actions within a Horizon generated from multiple policies

Solution:

Decoupled PPO Objective

- Weigh actions in a trajectory by the distance between the action generation policy and the current policy

Did this work? 😊

I hope it did → Evaluations/Results

Evaluation/Results

Models:

- R1-Distilled-Qwen models (1.5B to 32B)

Datasets:

- AIME24 (math)
- LiveCodeBench (coding)

Baselines:

Synchronous RL systems

- DeepScaleR (math, 1.5B)
- DeepCoder (coding, 14B)
- Synchronous AReal (others)



Results

AReal achieves comparable performance with 2× fewer training hours

Model	AIME24 ↑	# Nodes	PPO Steps	Training Hours ↓
1.5B basemodel	29.3	-	-	-
w/ VeRL	43.1*	16	250	33.6
w/ Sync.AReaL	42.0	16	250	41.0
w/ AReaL (ours)	42.2	16	250	14.8
7B basemodel	54.3	-	-	-
w/ VeRL	-	24	250	52.1
w/ Sync.AReaL	63.0	24	250	57.7
w/ AReaL (ours)	63.1	24	250	25.4

Model	LiveCodeBench ↑	# Nodes	PPO Steps	Training Hours ↓
14B basemodel	53.4	-	-	-
w/ VeRL	57.9*	32	80	44.4
w/ Sync.AReaL	56.7	32	80	48.8
w/ AReaL (ours)	58.1	32	80	21.9
32B basemodel	57.4	-	-	-
w/ VeRL	-	48	60	46.4
w/ Sync.AReaL	61.2	48	60	51.1
w/ AReaL (ours)	61.0	48	60	31.1



Results

Scalability:

- AREAL shows approximate linear scaling trend with increased device count, while
- synchronous system typically fails to scale effectively

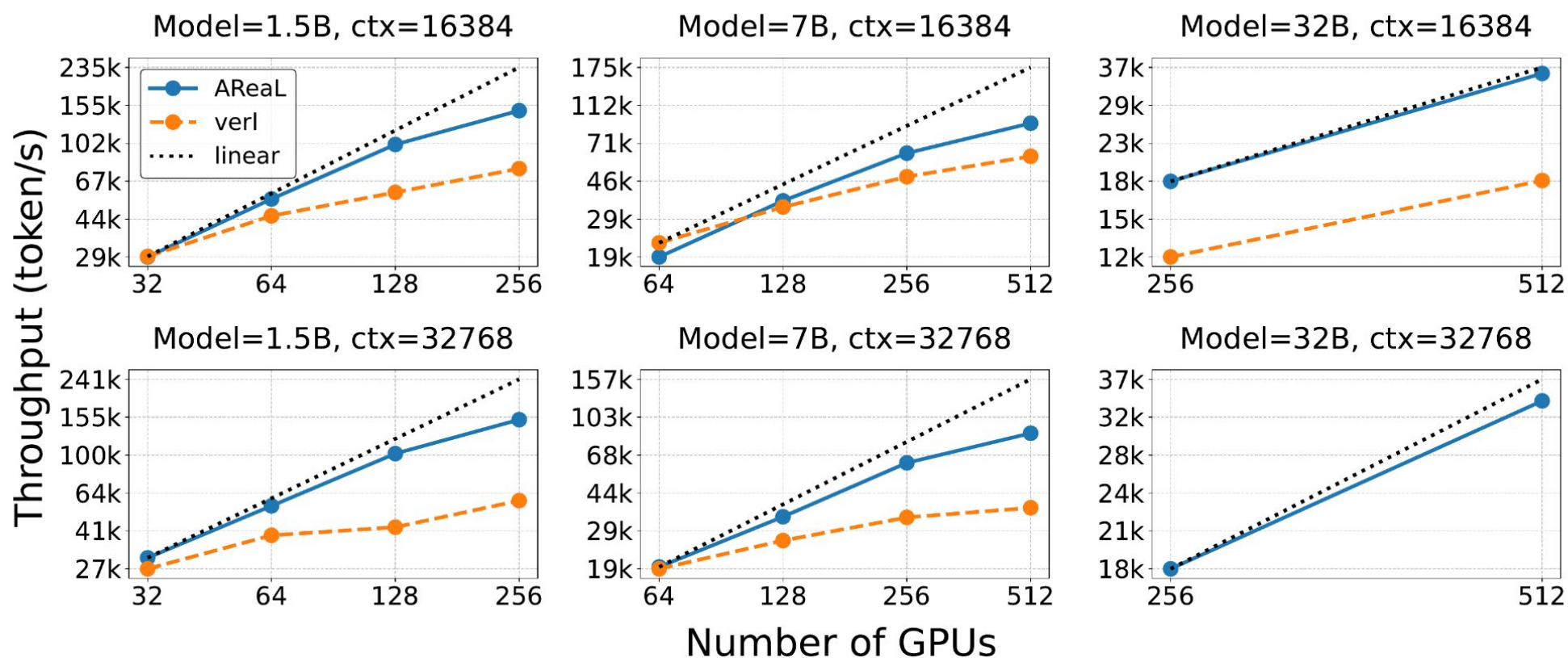
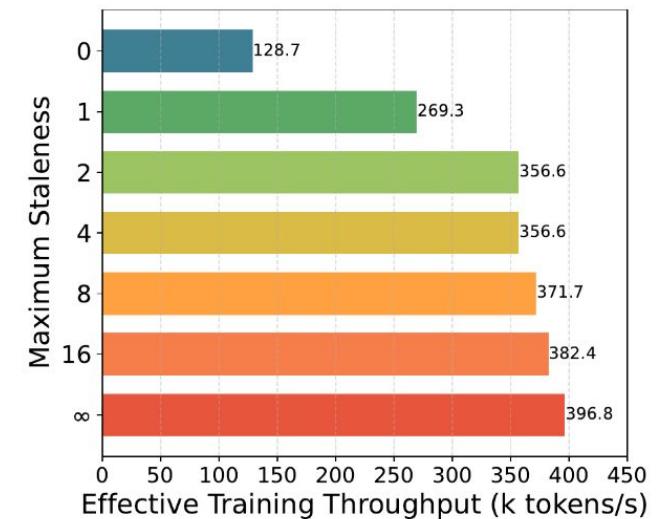
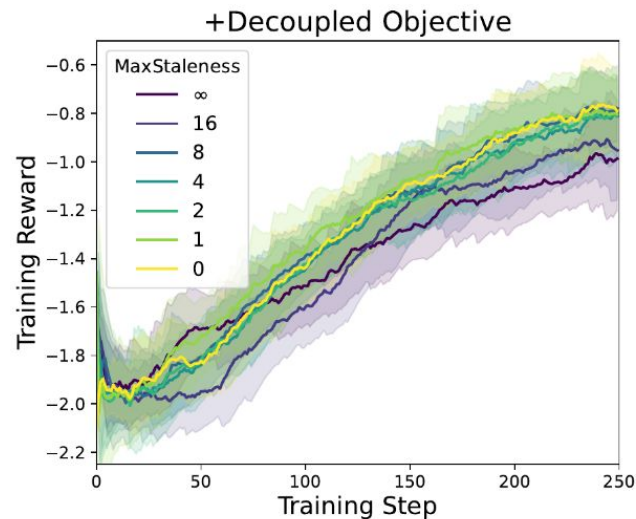
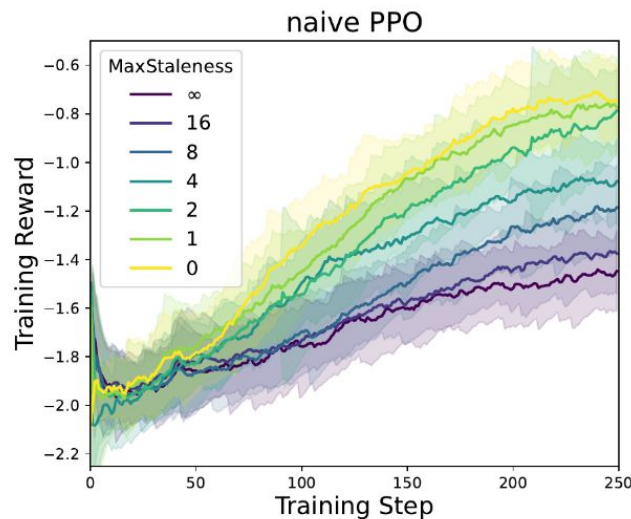


Figure 4: The strong scaling trend. Dotted lines indicate ideal linear scaling. verl consistently encounters OOM with 32k context length and the 32B model so the data points are missing.

Ablations



(a) Learning curves with naive PPO.

(b) Learning curves with eq. (5).

(c) Effective training throughput.

Figure 5: Ablation studies of the decoupled PPO objective and staleness control with a 1.5B model on math reasoning tasks. Both algorithmic choices are essential. With a moderate staleness value and the decoupled objective, training progress can be accelerated by over $2\times$ while maintaining final evaluation performance.

- Decoupled objective allows for smooth learning curve
- Allowing Staleness allows for more effective training throughput

Ablations

Table 2: Evaluation scores when varying data staleness, comparing performance with and without the decoupled objective. Numbers within ± 1 of the oracle score are underlined.

Max.Stale.	AIME24		AIME25		AMC23		MATH 500	
	W/o	With	W/o	With	W/o	With	W/o	With
0 (Oracle)	42.0		32.9		84.4		89.2	
1	<u>41.8</u>	<u>42.1</u>	30.7	<u>31.9</u>	<u>83.3</u>	<u>85.2</u>	<u>89.9</u>	<u>89.8</u>
2	40.0	<u>41.8</u>	<u>32.1</u>	<u>32.5</u>	82.3	<u>84.3</u>	<u>89.6</u>	<u>89.6</u>
4	23.3	<u>42.2</u>	23.1	<u>32.0</u>	58.5	<u>85.1</u>	66.9	<u>89.5</u>
8	35.7	<u>41.0</u>	27.8	<u>31.1</u>	81.2	82.9	87.8	<u>89.2</u>
16	35.8	38.7	26.2	<u>32.5</u>	78.4	83.2	87.4	<u>89.1</u>
∞	34.0	36.9	26.9	29.9	79.4	81.0	87.1	88.1

- There is a threshold n above which there is significant performance degradation even with decoupled objective

Ablations

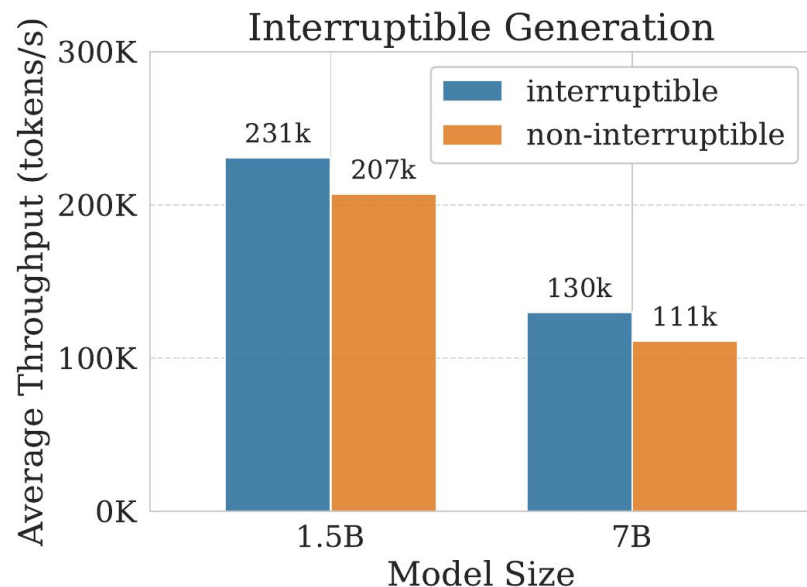


Figure 6: Ablation study of interruptible generation.

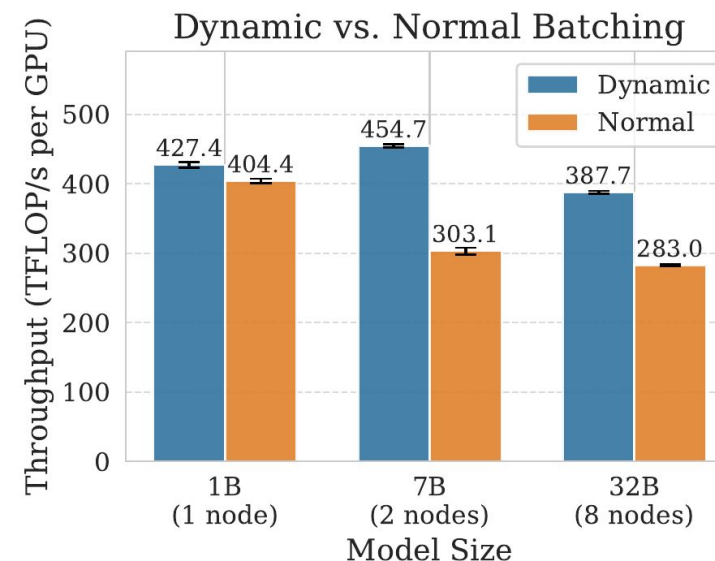


Figure 7: Ablation study of dynamic micro-batch allocation.

- Interruptible generation can lead to 12-17% throughput increase
- Dynamic batching on average can yield 30% throughput improvements across various model sizes

Conclusion

Yay! AReal works 😊 🙌

Limitations/Weakness

- **One-step parallel setup not part of the baselines**
- **Reproducibility might be an issue beyond their experimental setting.**

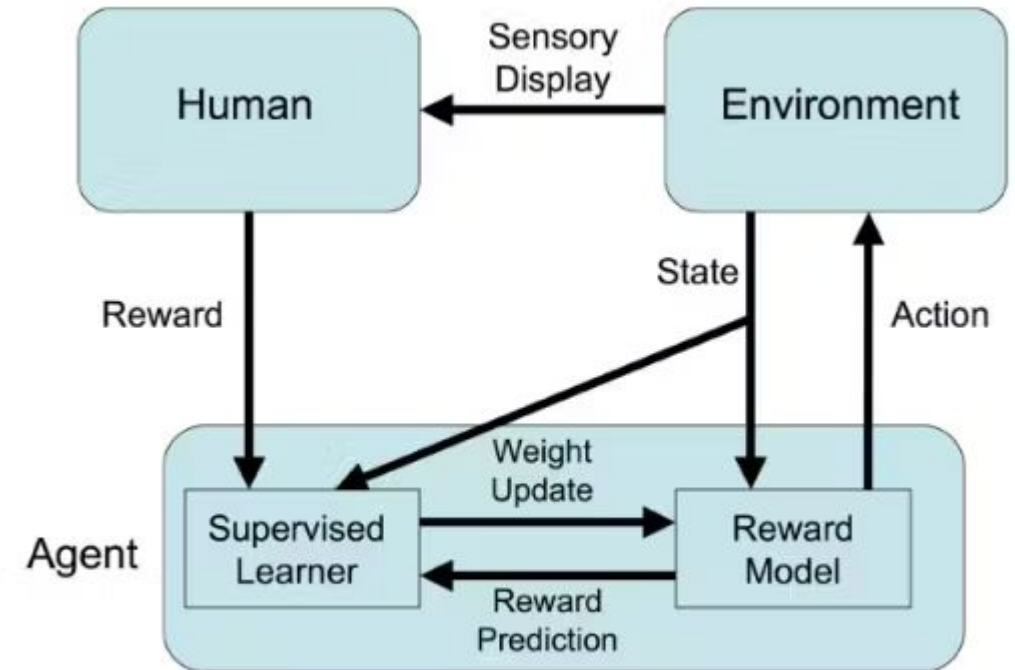
Future Work

- **Idle moments? How about we do some dynamic resource allocations**

HybridFlow: A Flexible and Efficient RLHF Framework

RLHF

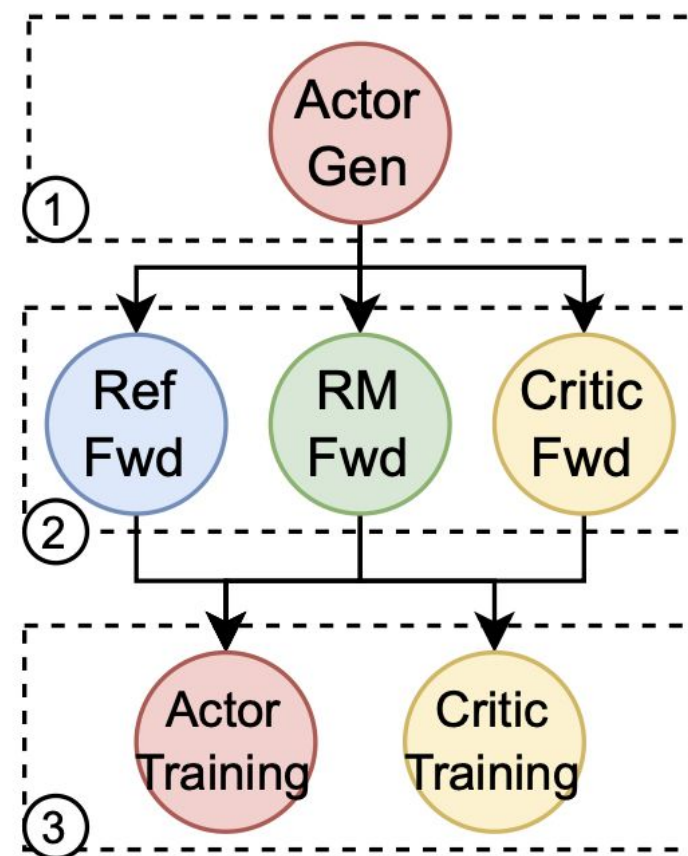
- Reinforcement Learning from Human Feedback
- Align LLM to human values



Example of reinforcement learning from human feedback (Image source: [cs.utexas.edu](https://www.cs.utexas.edu/~bradknox/papers/icdl08-knox.pdf))

Traditional RLHF System

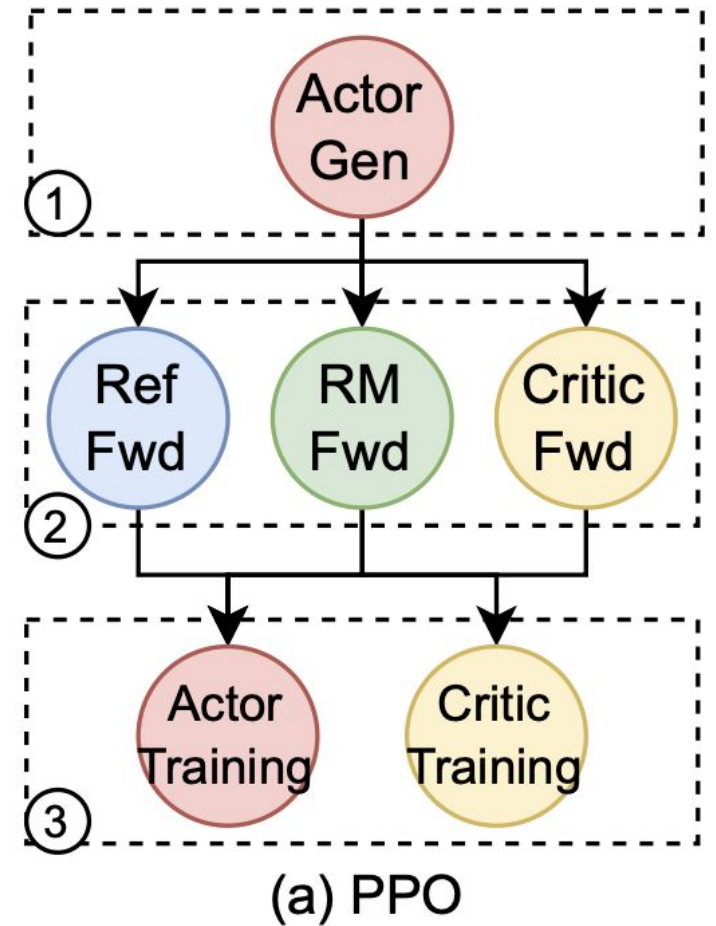
- Proximal Policy Optimization
- Four LLMs
 - Actor
 - Critic
 - Reference policy
 - Reward model
- Three stages
 - Generation
 - Preparation
 - Training



(a) PPO

RLHF Dataflow

- Traditional RL dataflow is a directed acyclic graph
- RLHF dataflow is more complex
 - Complex models
 - Different computations
 - Complex data dependency
 - Different parallelism per model
- Many different RLHF algorithms



Existing Parallelism Strategies

- Traditional RL uses a **single-controller** system
 - Centralized controller controls node processes, execution order
 - Controller generates distributed workers for a single process
 - Constrained to small NNs (100s MB) because...
 - Too much communication overhead for coordination messages to workers in big RLHF systems
- Existing RLHF uses a **multi-controller** system
 - Each controller controls one device
 - Every controller communicates with others with little overhead
 - However, very rigid with no central control
 - Nested, complicated code

What characterizes RLHF systems?

- Heterogeneous model workloads
 - Actor, critic, reference, reward models execute training, inference, generation at different stages
 - Different parallelism, optimization for each model
- Unbalanced training vs generation computation
 - Different parallelism strategies for actor training/generation optimizes throughput...
 - But resharding between stages at runtime causes big overhead
- Strategic device placement of models
 - Models put on different GPUs can run in parallel... but may have to idle
 - Models put on the same GPU reduce idle time... but must run sequentially
 - Good system should support flexible placement

Existing RLHF Systems

- Heterogeneous model workloads
 - Actor, critic, reference, reward models execute training, inference, generation at different stages
 - Different parallelism, optimization for each model

Existing RLHF Systems

- Strategic device placement of models
 - Models put on different GPUs can run in parallel... but may have to idle
 - Models put on the same GPU reduce idle time... but must run sequentially
 - Good system should support flexible placement

RLHF system	DeepSpeed-Chat	OpenRLHF	NeMo-Aligner	HybridFlow
Parallelism	Training: ZeRO Generation:TP	Training: ZeRO Generation:TP	3D Parallelism for both training and generation	Training: 3D, ZeRO, FSDP Generation: 3D Parallelism
Actor weights in training & generation	Model resharding from ZeRO to TP	Using two copies of actor weights for the two stages	Using identical model partition in two stages (shared weights)	Zero-redundancy model resharding
Model Placement	Colocate all models on the same set of devices	Each model placed on separate devices	Actor/Ref colocated on some GPUs Critic/RM colocated on other GPUs	Support various model placement
Execution Pattern Actor Critic Reference Policy GPU Process Reward model				Support various execution patterns

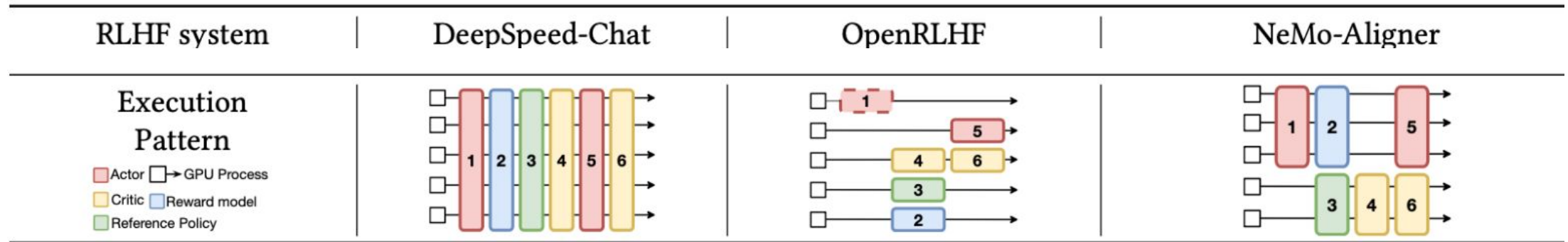
Existing RLHF Systems

- Unbalanced training vs generation computation
 - Different parallelism strategies for actor training/generation optimizes throughput
 - ...But resharding between stages at runtime causes big overhead

RLHF system	DeepSpeed-Chat	OpenRLHF	NeMo-Aligner	HybridFlow
Parallelism	Training: ZeRO Generation: TP	Training: ZeRO Generation: TP	3D Parallelism for both training and generation	Training: 3D, ZeRO, FSDP Generation: 3D Parallelism
Actor weights in training & generation	Model resharding from ZeRO to TP	Using two copies of actor weights for the two stages	Using identical model partition in two stages (shared weights)	Zero-redundancy model resharding
Model Placement	Colocate all models on the same set of devices	Each model placed on separate devices	Actor/Ref colocated on some GPUs Critic/RM colocated on other GPUs	Support various model placement
Execution Pattern Actor GPU Process Critic Reward model Reference Policy				Support various execution patterns

Existing RLHF Systems: Inefficient

- Existing frameworks are limited to one model placement plan...
- ... one execution pattern!
- Unbalanced model workload -> Inefficient utilization



Actor GPU Process

Critic Reward model

Reference Policy

Existing RLHF Systems: Inflexible

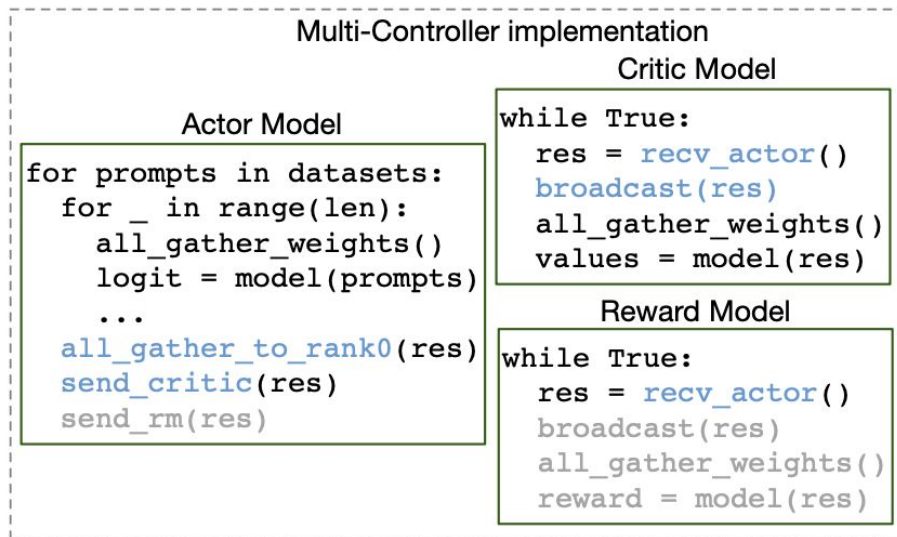
- Multi-controller structure ties together communication, computation, data transfer code
- Not modular, reusable
- Parallelism strategies take significant effort
- Changing parallelism, algorithmic strategies -> re-implement whole system



Solution: HybridFlow

- Hierarchical **hybrid** programming model for RLHF data**flow**
- Single-controller system at **inter-node** level
 - Flexibility in coordinating data, execution order communication
 - RLHF dataflow has only a few nodes/models
- Multi-controller system at **intra-node** level
 - Fast at enabling distributed computation, per model
- Combine both for flexibility *and* efficiency
- Low overhead



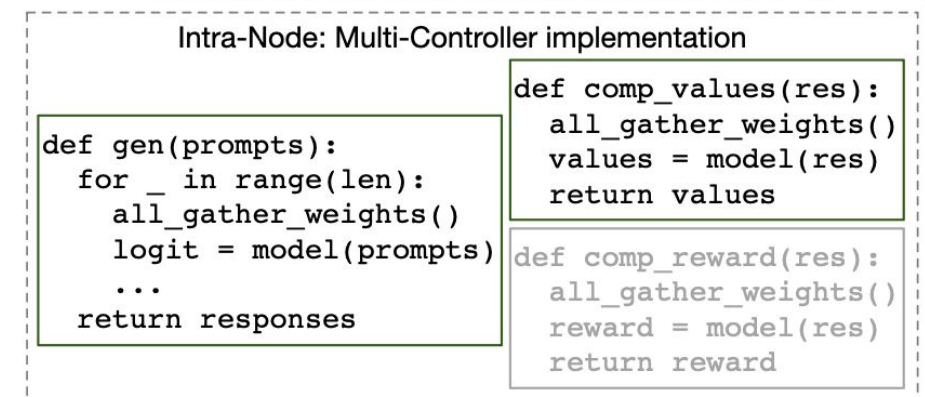
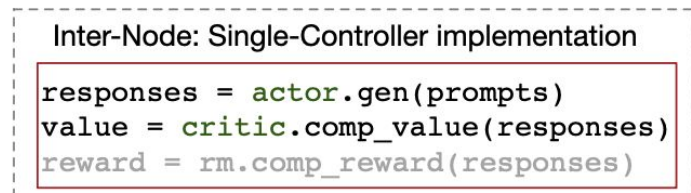


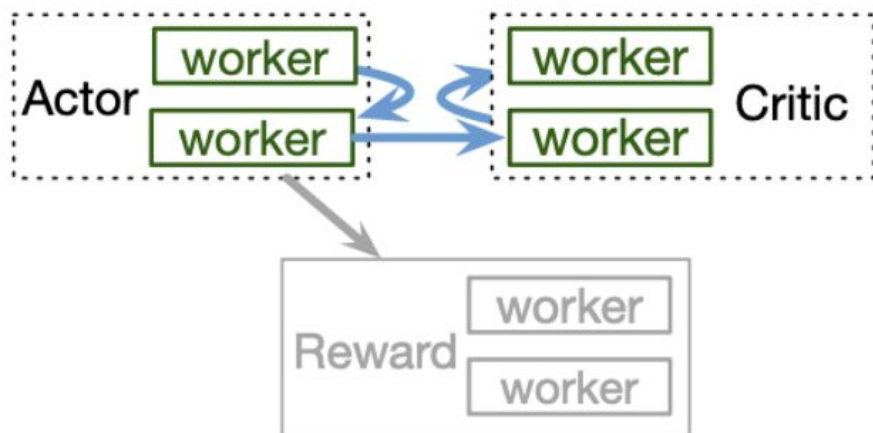
Existing RLHF frameworks

- Separate code for each model and its workers
- Workers only have local view of system state, must handle communication themselves
- Computation & data transfer code mixed

HybridFlow

- Workers don't have to handle communication themselves
- Computation & data transfer code separated
- Single controller process to handle communication & data transfer



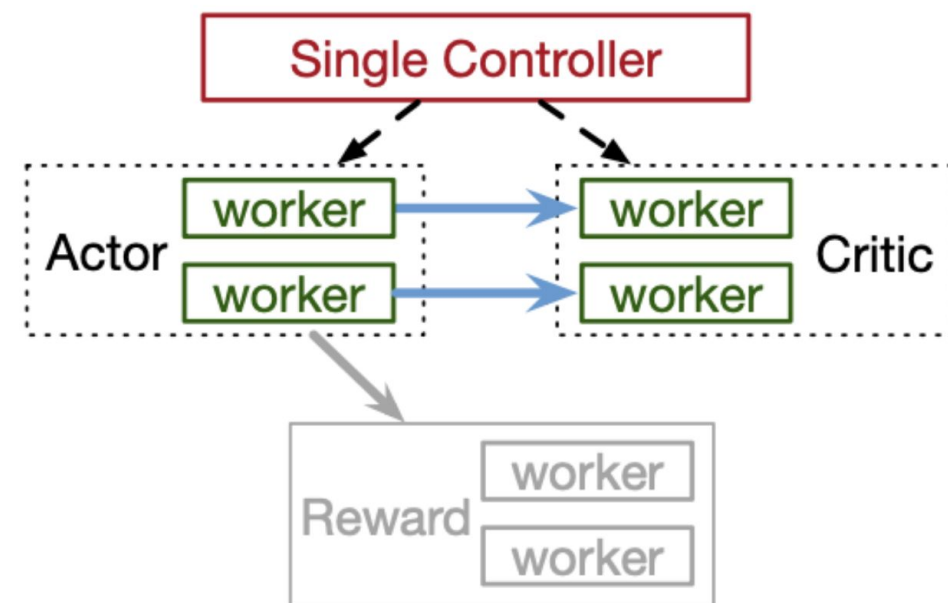


Existing RLHF frameworks

- Communication needs to happen between workers and between models
- Local computation & all gather between workers, and send/receive to other models

HybridFlow

- Communication simplified
- Local computation separated from data transfer





Existing RLHF frameworks

- Model placement is rigid per framework
- Large transition overhead when going from training to generation
- Mostly coupled tightly to PPO algorithm

HybridFlow

- Can support any algorithm
- No redundancy when transitioning from training to generation
- Different model placement strategies are supported



HybridFlow Architecture

- How to provide flexibility for users?



hierarchical APIs

Unparalleled code reuse

```
# Initialize cost model by reusing the RewardWorker
cost = RewardWorker(cost_config, resource_pool)
... # omit other models initialization
algo_type = "Safe-RLHF" # specify different RLHF numerical computation.
# Examples of PPO and Safe-RLHF
for (prompts, pretrain_batch) in dataloader:
    # Stage 1: Generate responses
    batch = actor.generate_sequences(prompts)
    batch = actor.generate_sequences(prompts, do_sample=False)
    # Stage 2: Prepare experience
    batch = critic.compute_values(batch)
    batch = reference.compute_log_prob(batch)
    batch = reward.compute_reward(batch)
    batch = cost.compute_cost(batch)
    batch = compute_advantages(batch, algo_type)
    # Stage 3: Actor and critic training
    critic_metrics = critic.update_critic(batch, loss_func=algo_type)
    pretrain_loss = actor.compute_loss(pretrain_batch)
    batch["pretrain_loss"] = pretrain_loss
    actor_metrics = actor.update_actor(batch, loss_func=algo_type)
```

Annotations in the code:

- `cost = RewardWorker(cost_config, resource_pool)`: is added for ReMax
- `batch = actor.generate_sequences(prompts, do_sample=False)`: Not necessary in ReMax
- `batch = cost.compute_cost(batch)`: is added for Safe-RLHF

Figure 6. Implementation of PPO [55], ReMax [43], and Safe-RLHF [19]. Users can adapt to different RLHF algorithms by simply adding or deleting a few lines of code.

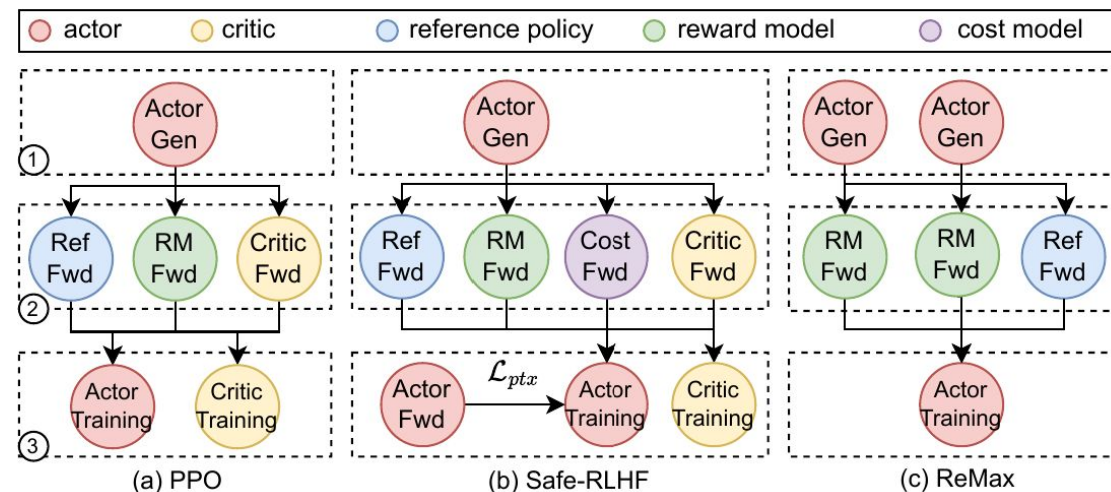


Figure 1. Dataflow graph of 3 RLHF algorithms [19, 43, 55]. Stage ①, ②, ③ represent Generation, Preparation, and Training, respectively.

HybridFlow Architecture

- How to provide flexibility for users?
- Actor training and generation: how to minimize redundancy?
- How can it determine an optimal device placement of each model to maximize throughput?



hierarchical APIs

Hierarchical APIs and Data Resharding

- Hierarchical APIs encapsulate distributed RLHF
- Single controller orchestrates multi-model flow
- Workers manage local 3D parallel groups
- Transfer protocols handle cross-model communication
- Asynchronous resharding overlaps compute, transfers
- Decouples computation, communication for flexibility

```
class ActorWorker(3DParallelWorker):  
    # An example of distributed computation function  
    @register(transfer_mode=3D_PROTO)  
    def update_actor(self, prompts: DataProto):  
        ...  
# Allocate devices for a ResourcePool  
resource_pool = ResourcePool([n_gpus_per_machine] * n_machines)  
# Map the model to allocated devices and init model  
actor_model = ActorWorker(actor_config, resource_pool)
```

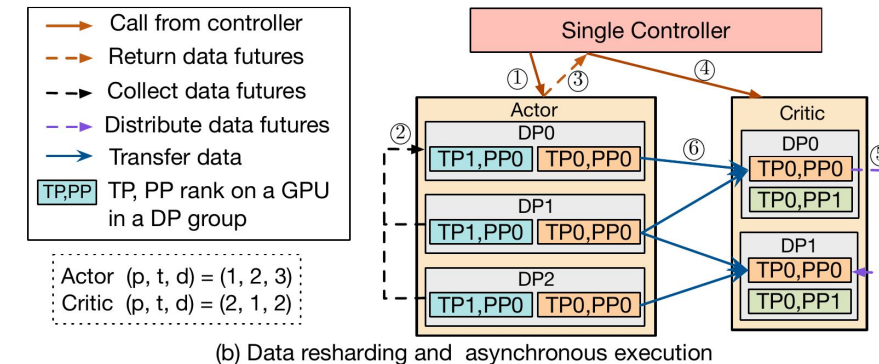
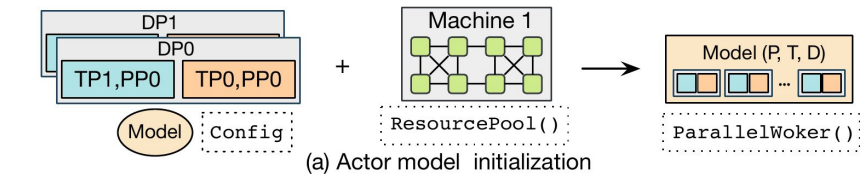


Figure 5. An illustration of hierarchical APIs. (a) Model with 3D parallel configuration, resource allocation, and 3DParallelWorker initialization. (b) Asynchronous data resharding between two models with collect and distribute functions in 3D_PROTO.

3D-HybridEngine Workflow

- Actor alternates training and generation phases
- Different 3D groups for each phase
- Seamless weight transitions without duplication
- Overlaps forward, backward, and communication tasks
- Increases throughput, reduces synchronization delays

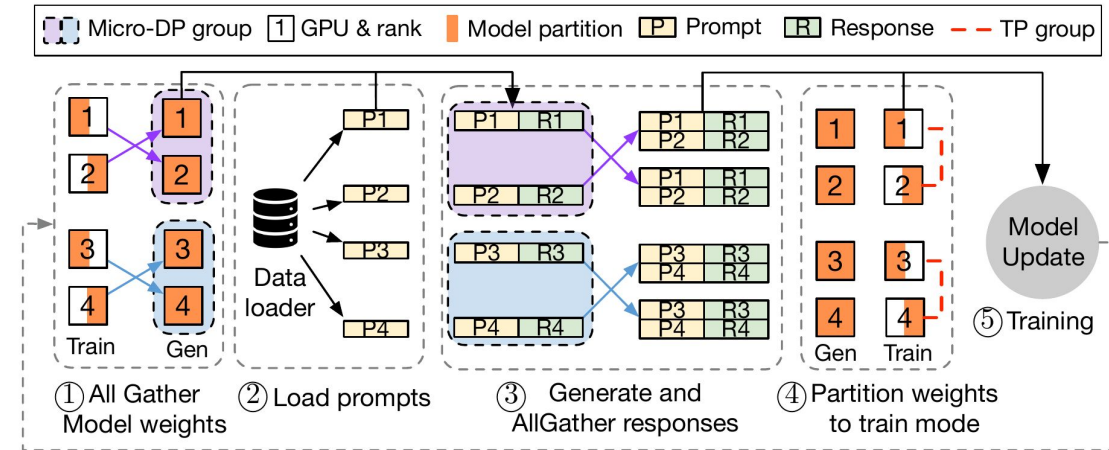


Figure 7. 3D-HybridEngine workflow in one RLHF iteration. 4 GPUs are used for actor training and generation. 1-2-2 (p - t - d) parallel groups are used in training and 1-1-2-2 (p_g - t_g - d_g - d) parallel groups are used in generation.

Zero-Redundancy Model Resharding

- Compares vanilla versus optimized resharding groups
- HybridFlow aligns training-generation GPU mappings
- Reuses weights, avoids redundant memory storage
- Minimizes AllGather communication during reshaping
- Achieves zero-redundancy parameter synchronization design

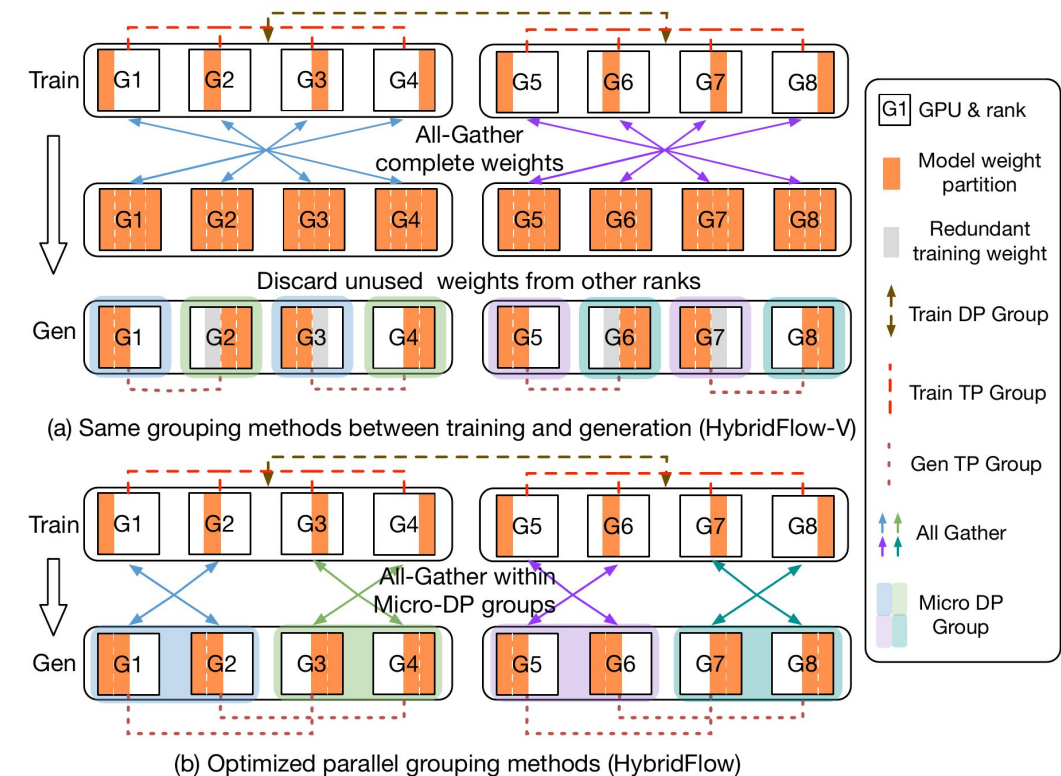


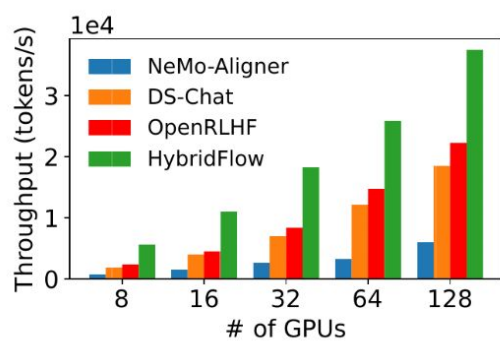
Figure 8. Model weights resharding. 2 machines each with 4 GPUs are used for actor training and generation.

Transition overhead

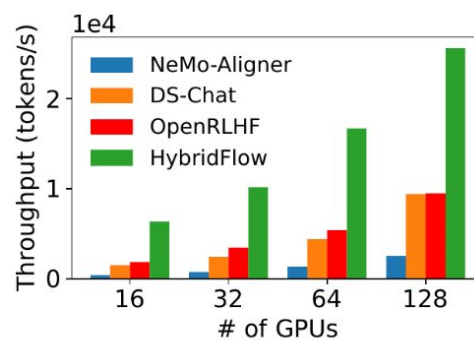
- DS-Chat: full memory duplication
- HybridFlow-V: less communication, but still redundant weights
- HybridFlow: minimal communication overhead

Table 2. Transition overhead between training & generation

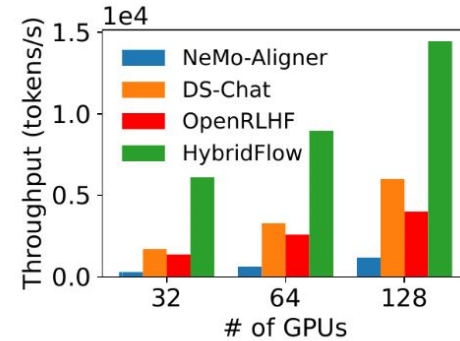
	DS-Chat	HybridFlow-V	HybridFlow
Comm. Vol	$\frac{tpd-1}{tpd}M$	$\frac{tp-1}{tp}M$	$\frac{tp-t_gp_g}{t_gp_gtp}M$
Peak Mem.	M	M	$\frac{1}{t_gp_g}M$
Redundancy	$\frac{1}{tpd}M$	$\frac{1}{tp}M$	0



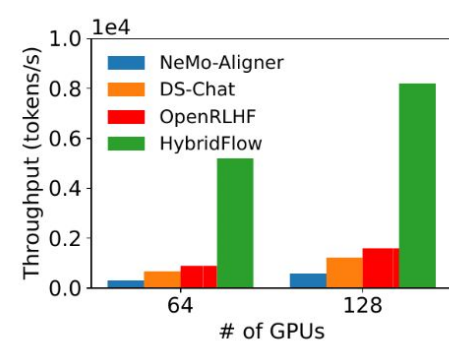
(a) 7B ($1.68\times\sim8.63\times$)



(b) 13B ($2.70\times\sim18.96\times$)

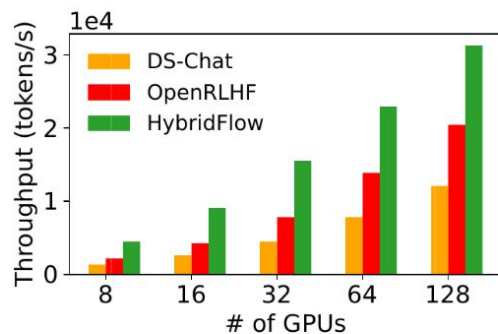


(c) 34B ($2.41\times\sim20.57\times$)

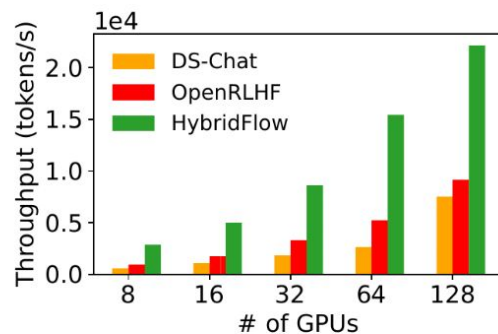


(d) 70B ($5.17\times\sim17.98\times$)

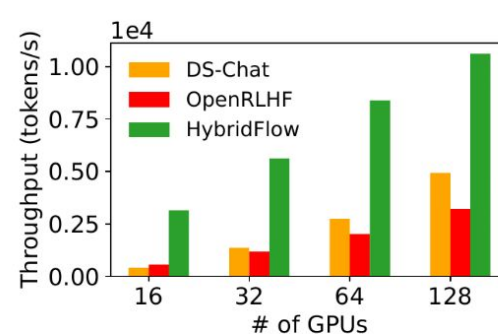
Figure 9. PPO throughput. Numbers in parentheses are HybridFlow speedups compared with baselines.



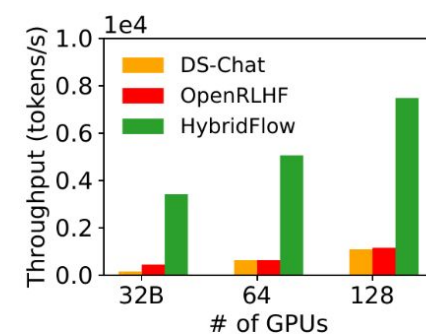
(a) 7B ($1.53\times\sim2.56\times$)



(b) 13B ($2.49\times\sim3.66\times$)

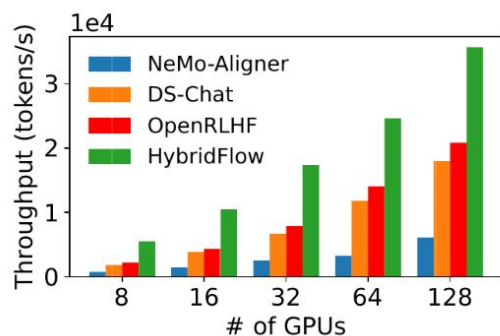


(c) 34B ($2.14\times\sim4.80\times$)

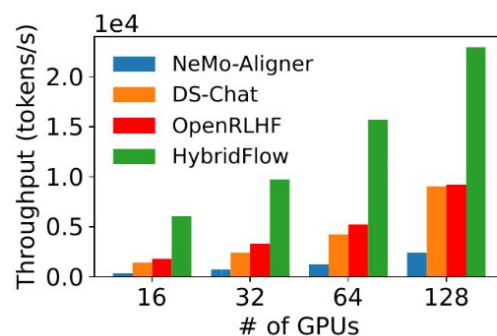


(d) 70B ($6.46\times\sim9.78\times$)

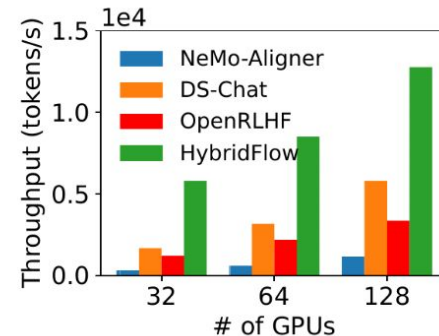
Figure 10. ReMax throughput. Numbers in parentheses are HybridFlow speedups compared with baselines



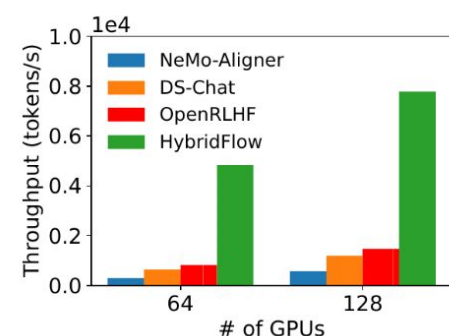
(a) 7B ($1.71\times\sim12.87\times$)



(b) 13B ($2.49\times\sim18.47\times$)



(c) 34B ($2.20\times\sim19.76\times$)



(d) 70B ($4.89\times\sim16.86\times$)

Figure 11. Safe-RLHF throughput. Numbers in the parentheses are HybridFlow speedups compared with the baselines

Evaluations

Best throughput

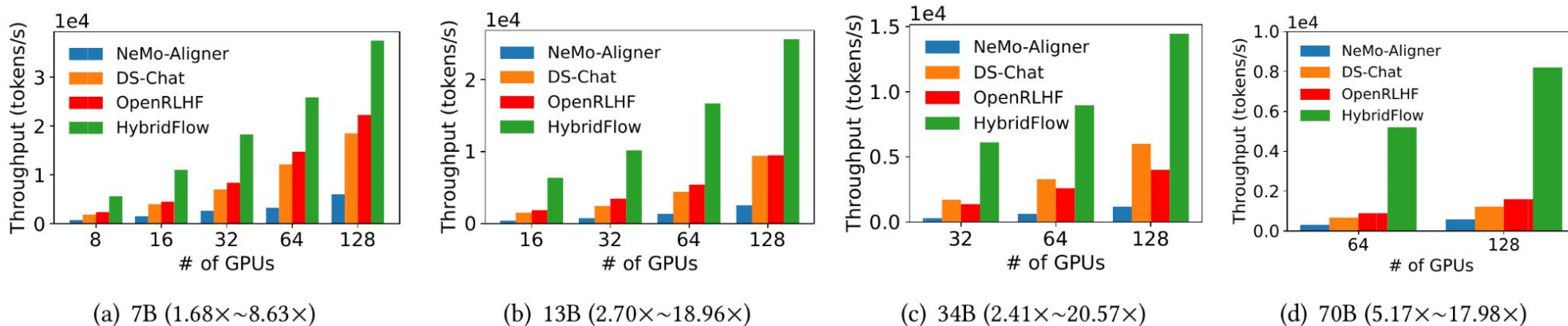
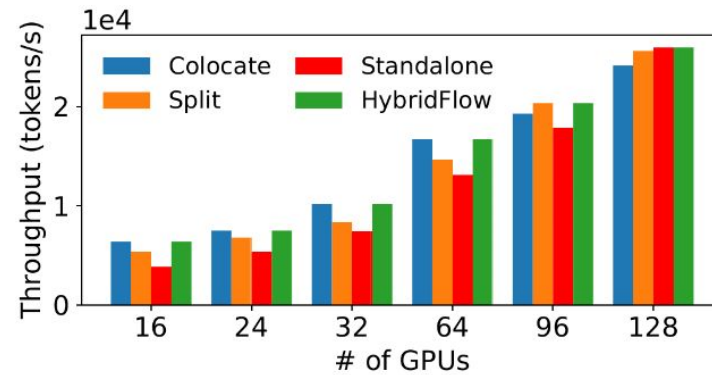


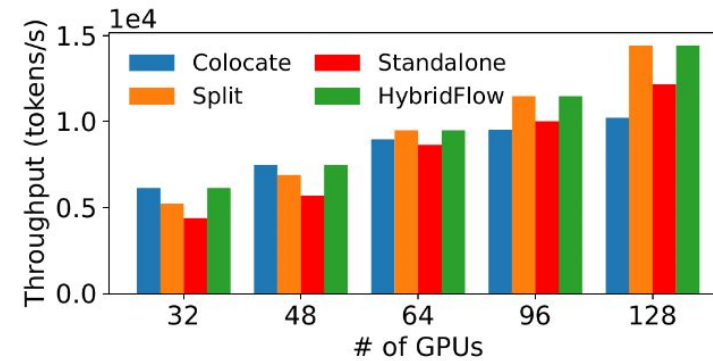
Figure 9. PPO throughput. Numbers in parentheses are HybridFlow speedups compared with baselines.

*Same pattern for ReMax and Safe-RLHF

Best placement in all cases



(a) 13B



(b) 34B

Figure 12. Throughput of HybridFlow under different placements

Transition time

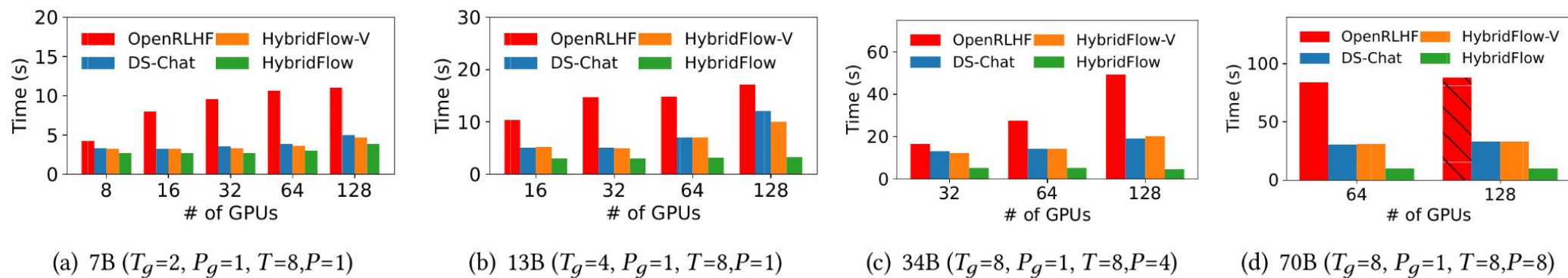


Figure 14. Transition time between actor training and generation.

Generation optimization and algorithm runtime

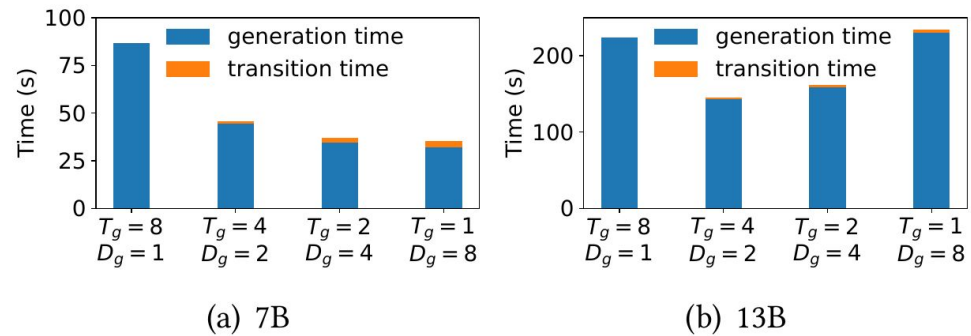


Figure 15. Time breakdown on different generation parallel sizes of the actor model on 16 GPUs.

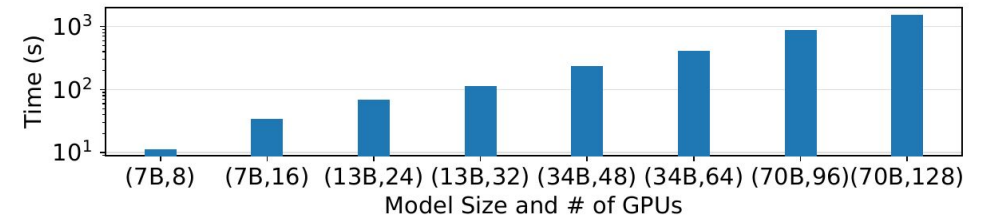


Figure 16. Runtime of device mapping algorithm. The model size and # of GPUs are simultaneously scaled.

Conclusion

HybridFlow integrates single and multi controller paradigms to eliminate coordination bottlenecks

It enables zero-redundancy resharding and modular RLHF development, yet it still allows for up to 20x faster large scale LLM alignment



Limitations

“Algorithm 1 can be readily
extended for optimizing model mapping over heterogeneous
devices”

Recap

- **AReal**
 - Designed to scale **RL** systems for reasoning
 - **Key:** Decoupling training from generation
- **HybridFlow**
 - Designed to scale **RLHF** systems
 - **Key:** Hybrid single & multi-controller paradigm for dataflow

Thank you!

